

Funzioni

Funzioni in C++

Come definire e usare le funzioni in C++.

Cos'è una funzione?

Immagina di dover calcolare l'area di un cerchio in dieci punti diversi del tuo programma. Scriveresti la stessa formula dieci volte? E se poi vuoi cambiarla, la aggiorni in dieci posti?

Le **funzioni** risolvono questo problema: scrivi il codice una volta, gli dai un nome, e lo richiami ogni volta che ne hai bisogno. È come salvare una ricetta: la scrivi una volta, poi la segui ogni volta che vuoi cucinare quel piatto.

Definire una funzione

Una funzione si scrive così:

```
tipo_ritorno nomeFunzione(tipo parametro1, tipo parametro2) {  
    // codice della funzione  
    return valore; // se il tipo di ritorno non è void  
}
```

La funzione più semplice: non riceve nulla e non restituisce nulla:

```
void saluta() {  
    cout << "Ciao!" << endl;  
}
```

void significa "niente": questa funzione non restituisce nessun valore.

Chiamare una funzione

Una volta definita, puoi "chiamarla" (cioè eseguirla) scrivendo il suo nome seguito dalle parentesi:

```
void saluta() {  
    cout << "Ciao!" << endl;  
}  
  
int main() {  
    saluta();    // esegue la funzione – stampa "Ciao!"  
    saluta();    // la puoi chiamare quante volte vuoi  
    return 0;  
}
```

Passare informazioni: i parametri

I **parametri** sono informazioni che puoi passare alla funzione quando la chiami:

```
// Questa funzione riceve un nome e lo usa per salutare  
void saluta(string nome) {  
    cout << "Ciao, " << nome << "!" << endl;  
}  
  
int main() {  
    saluta("Alice");    // stampa: Ciao, Alice!  
    saluta("Bob");      // stampa: Ciao, Bob!  
    return 0;  
}
```

Puoi avere più parametri separati da virgola:

```
// Riceve base e altezza, calcola area e perimetro
void stampaRettangolo(int base, int altezza) {
    cout << "Area: " << base * altezza << endl;
    cout << "Perimetro: " << 2 * (base + altezza) << endl;
}

int main() {
    stampaRettangolo(5, 3);
    stampaRettangolo(10, 7);
    return 0;
}
```

Ricevere un risultato: il valore di ritorno

Una funzione può anche **restituire** un valore al codice che la ha chiamata. Si usa `return` :

```
// Riceve due numeri e restituisce la loro somma
int somma(int a, int b) {
    return a + b;
}

int main() {
    int risultato = somma(3, 5);
    cout << risultato << endl;    // 8

    // Puoi usare la funzione direttamente nelle espressioni
    cout << somma(10, 20) * 2 << endl;    // 60

    return 0;
}
```

Quando il compilatore incontra `return` , la funzione si ferma subito e restituisce il valore.

Il prototipo: dichiarare prima di usare

In C++, una funzione deve essere **dichiarata prima del punto in cui la chiami**. Se vuoi definire le funzioni dopo il `main` , devi prima scrivere il loro **prototipo** (una dichiarazione anticipata):

```
// Prototipo – dice al compilatore che questa funzione esisterà
int somma(int a, int b);

int main() {
    cout << somma(3, 5) << endl;    // funziona!
    return 0;
}

// Definizione completa – scritta dopo il main
int somma(int a, int b) {
    return a + b;
}
```

Il prototipo ha la stessa firma della funzione ma termina con `;` invece del corpo.

Valori di default

Puoi dare un valore predefinito a un parametro. Se chi chiama la funzione non lo specifica, viene usato il valore di default:

```
void saluta(string nome, string saluto = "Ciao") {
    cout << saluto << ", " << nome << "!" << endl;
}

int main() {
    saluta("Alice");                // usa il default – stampa: Ciao, Alice!
    saluta("Bob", "Buongiorno");    // usa il valore specificato – stampa:
    Buongiorno, Bob!
    return 0;
}
```

I parametri con valori di default devono stare sempre in fondo alla lista.

Più istruzioni `return`

Una funzione può avere più punti di uscita. Appena il programma incontra `return`, la funzione si ferma:

```
string classificaVoto(int voto) {
    if (voto >= 9) return "Ottimo";
    if (voto >= 7) return "Buono";
    if (voto >= 6) return "Sufficiente";
    return "Insufficiente"; // raggiunto solo se nessun return precedente
    è scattato
}
```

Passaggio per valore: la funzione riceve una copia

Di default, una funzione riceve una **copia** del valore. Le modifiche che fa non cambiano la variabile originale:

```
void raddoppia(int x) {
    x = x * 2;    // modifica solo la copia - l'originale non cambia
    cout << "Dentro la funzione: " << x << endl;
}

int main() {
    int numero = 5;
    raddoppia(numero);
    cout << "Fuori dalla funzione: " << numero << endl; // ancora 5!
    return 0;
}
```

Output:

```
Dentro la funzione: 10
Fuori dalla funzione: 5
```

Per modificare la variabile originale, si usano i riferimenti (trattati nella sezione memoria).

Esempio pratico

```
#include <iostream>
using namespace std;

// Calcola l'area di un cerchio dato il raggio
double calcolaAreaCerchio(double raggio) {
    const double PI = 3.14159265358979;
    return PI * raggio * raggio;
}

// Restituisce true se il numero è pari, false altrimenti
bool isPari(int numero) {
    return numero % 2 == 0;
}

// Restituisce il maggiore dei due numeri
int massimo(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    double raggio = 5.0;
    cout << "Area cerchio (r=" << raggio << "): " <<
    calcolaAreaCerchio(raggio) << endl;

    int n = 7;
    cout << n << " è " << (isPari(n) ? "pari" : "dispari") << endl;

    cout << "Massimo tra 15 e 23: " << massimo(15, 23) << endl;

    return 0;
}
```

Overloading di Funzioni

Come definire più funzioni con lo stesso nome ma parametri diversi in C++.

Più versioni della stessa funzione

Immagina di voler calcolare l'area di forme diverse: un cerchio, un rettangolo, un triangolo. Potresti chiamare le funzioni `areaDelCerchio`, `areaDelRettangolo`, `areaDelTriangolo`. Ma sarebbe più comodo chiamarle tutte `area` e lasciare al compilatore il compito di capire quale usare.

In C++ questo si chiama **overloading** (sovraccarico): puoi avere più funzioni con lo stesso nome, purché abbiano parametri diversi.

Come funziona

Il compilatore distingue le funzioni guardando il numero e il tipo dei parametri. Quando chiami la funzione, sceglie automaticamente quella giusta:

```
void stampa(int x) {
    cout << "Intero: " << x << endl;
}

void stampa(double x) {
    cout << "Decimale: " << x << endl;
}

void stampa(string s) {
    cout << "Stringa: " << s << endl;
}

int main() {
    stampa(42);           // il compilatore sceglie stampa(int)
    stampa(3.14);        // il compilatore sceglie stampa(double)
    stampa("Ciao");       // il compilatore sceglie stampa(string)
    return 0;
}
```

Output:

```
Intero: 42
Decimale: 3.14
Stringa: Ciao
```

Overloading con numero diverso di parametri

Puoi anche avere versioni con un numero diverso di parametri:

```
int somma(int a, int b) {  
    return a + b;  
}  
  
int somma(int a, int b, int c) {  
    return a + b + c;  
}  
  
int somma(int a, int b, int c, int d) {  
    return a + b + c + d;  
}  
  
int main() {  
    cout << somma(1, 2) << endl;           // 3  
    cout << somma(1, 2, 3) << endl;        // 6  
    cout << somma(1, 2, 3, 4) << endl;     // 10  
    return 0;  
}
```

Esempio pratico: area di figure diverse

Un uso classico è calcolare l'area di figure geometriche diverse con lo stesso nome `area` :


```

#include <iostream>
using namespace std;

const double PI = 3.14159265358979;

// Area del cerchio – riceve solo il raggio
double area(double raggio) {
    return PI * raggio * raggio;
}

// Area del rettangolo – riceve base e altezza
double area(double base, double altezza) {
    return base * altezza;
}

// Area del triangolo – riceve base, altezza e un flag per distinguerla dal
// rettangolo
double area(double base, double altezza, bool triangolo) {
    return (base * altezza) / 2;
}

int main() {
    cout << "Area cerchio (r=5): " << area(5.0) << endl;
    cout << "Area rettangolo (4x6): " << area(4.0, 6.0) << endl;
    cout << "Area triangolo (3x8): " << area(3.0, 8.0, true) << endl;
    return 0;
}

```

Cosa non funziona: il tipo di ritorno non basta

Il compilatore distingue le funzioni solo dai parametri, non dal tipo di ritorno. Questo causa un errore:

```

int calcola(int x) { return x * 2; }
double calcola(int x) { return x * 2.0; } // ERRORE: i parametri sono
identici!

```

Il compilatore non sa quale delle due chiamare, e lo segnala come errore.

Overloading o parametri di default?

A volte si può ottenere lo stesso risultato con entrambi gli approcci:

```
// Con overloading – due funzioni separate
void saluta() { cout << "Ciao, ospite!" << endl; }
void saluta(string nome) { cout << "Ciao, " << nome << "!" << endl; }

// Con parametro di default – una sola funzione
void saluta(string nome = "ospite") {
    cout << "Ciao, " << nome << "!" << endl;
}
```

La regola generale:

- Usa i **parametri di default** quando la funzione fa la stessa cosa con valori diversi
- Usa l'**overloading** quando la funzione fa cose diverse per tipi diversi di input

Esempio pratico: confronto flessibile

```
#include <iostream>
#include <string>
using namespace std;

// Confronta due interi
bool confronta(int a, int b) {
    return a == b;
}

// Confronta due decimali con una piccola tolleranza
// (i numeri decimali raramente sono esattamente uguali)
bool confronta(double a, double b) {
    return (a - b < 0.0001) && (b - a < 0.0001);
}

// Confronta due stringhe
bool confronta(string a, string b) {
    return a == b;
}

int main() {
    cout << boolalpha;
    cout << confronta(5, 5) << endl;           // true
    cout << confronta(3.14, 3.14) << endl;       // true
    cout << confronta("ciao", "ciao") << endl;   // true
    cout << confronta("ciao", "Ciao") << endl;   // false (maiuscole
    contano)
    return 0;
}
```

Scope delle Variabili

La visibilità e la durata delle variabili in C++.

Cos'è lo scope?

Immagina un edificio con stanze. Quello che metti in una stanza non è visibile dalle altre stanze. Se esci dalla stanza, le cose che hai lasciato lì scompaiono.

In C++, le variabili funzionano allo stesso modo: esistono solo nel "posto" in cui vengono dichiarate, e spariscono quando quel posto termina.

Questo "posto" si chiama **scope** (visibilità).

Le parentesi graffe creano lo scope

In C++, ogni coppia di `{ }` crea una nuova zona di codice con il proprio scope. Le variabili dichiarate dentro quella zona esistono solo lì:

```
int main() {  
    int x = 10;  
  
    {  
        int y = 20;    // y esiste solo in questo blocco interno  
        cout << x;     // OK - x è visibile anche qui  
        cout << y;     // OK - y è visibile qui  
    }  
  
    cout << x;         // OK - x esiste ancora  
    // cout << y; // ERRORE - y non esiste più, il blocco è finito  
}
```

Variabili locali: vivono dentro la funzione

Una **variabile locale** è dichiarata dentro una funzione. Esiste solo finché quella funzione è in esecuzione, e non è visibile dall'esterno:

```

void miaFunzione() {
    int x = 10;    // x esiste solo dentro miaFunzione
    cout << x;    // OK
}

int main() {
    miaFunzione();
    // cout << x;  // ERRORE - x non esiste qui
    return 0;
}

```

Ogni chiamata alla funzione crea una nuova copia delle variabili locali, che poi sparisce quando la funzione termina.

Variabili globali: visibili ovunque

Una **variabile globale** è dichiarata fuori da qualsiasi funzione. È visibile da tutto il codice nel file:

```

int contatore = 0;    // variabile globale - visibile ovunque

void incrementa() {
    contatore++;      // può accedere e modificare la variabile globale
}

int main() {
    cout << contatore << endl;  // 0
    incrementa();
    incrementa();
    cout << contatore << endl;  // 2
    return 0;
}

```

Usa le variabili globali con cautela. Il fatto che qualsiasi funzione possa modificarle le rende difficili da controllare nei programmi grandi.

Le variabili dei cicli

Le variabili dichiarate dentro un `for` esistono solo per la durata del ciclo:

```
for (int i = 0; i < 5; i++) {  
    cout << i << " ";    // OK  
}  
// cout << i;    // ERRORE – i non esiste più fuori dal ciclo
```

Questo è un vantaggio: il contatore `i` non inquina il resto del codice.

Shadowing: quando due variabili hanno lo stesso nome

Se dichiari una variabile locale con lo stesso nome di una globale, la locale "nasconde" la globale dentro il suo scope. Si chiama **shadowing**:

```
int x = 100;    // variabile globale  
  
void funzione() {  
    int x = 50;    // variabile locale – nasconde quella globale  
    cout << x;    // stampa 50 (usa la locale)  
}  
  
int main() {  
    cout << x;    // stampa 100 (usa la globale)  
    funzione();  
    cout << x;    // stampa 100 (la globale non è cambiata)  
    return 0;  
}
```

Evita lo shadowing: usa nomi diversi per variabili diverse. Il codice è molto più chiaro.

La variabile `static` : sopravvive tra le chiamate

Normalmente, una variabile locale ricomincia da zero ogni volta che la funzione viene chiamata. Con la parola chiave `static`, il valore viene conservato tra una chiamata e l'altra:

```

void contaChiamate() {
    int locale = 0;           // ricomincia da 0 ad ogni chiamata
    static int statica = 0;   // conserva il valore tra le chiamate

    locale++;
    statica++;

    cout << "Locale: " << locale << ", Statica: " << statica << endl;
}

int main() {
    contaChiamate(); // Locale: 1, Statica: 1
    contaChiamate(); // Locale: 1, Statica: 2
    contaChiamate(); // Locale: 1, Statica: 3
    return 0;
}

```

I parametri delle funzioni

I parametri di una funzione si comportano come variabili locali: esistono solo dentro la funzione.

```

void funzione(int a, int b) {
    // a e b sono locali a questa funzione
    int somma = a + b;
    cout << somma;
}
// a, b e somma non esistono qui

```

Consigli pratici

- **Dichiara le variabili vicino a dove le usi:** non in cima alla funzione se le usi solo alla fine.
- **Evita le variabili globali** quando possibile. Preferisci passare i valori come parametri.
- **Evita lo shadowing:** dai nomi diversi alle variabili in scope diversi.

```
// Meno chiaro: variabile dichiarata lontano dall'uso
int risultato;
// ... molte righe di codice ...
risultato = a + b;

// Più chiaro: dichiarazione e uso insieme
int risultato = a + b;
```